

**Autor:** Thiago Marques Ramalho

**A Visão do Projeto** O desenvolvimento do Job Hunter AI foi um trabalho em equipe entre mim e a Inteligência Artificial. Enquanto a IA atuava como minha "digitadora" e assistente de código, o papel de planejar o fluxo, identificar os gargalos e criar as soluções reais foi totalmente humano. Abaixo, detalho os principais problemas que encontrei durante os meus testes práticos e como instruí a IA a resolvê-los para criar um sistema à prova de falhas.

### 1. O Problema da Lentidão no Site (Gupy)

- **O que eu percebi testando:** A Gupy é uma plataforma pesada. Quando o robô tentava ler a tela para procurar as vagas, a página demorava a carregar os botões, e o robô acabava se perdendo ou quebrando antes da hora.
- **A minha solução:** Percebi que tentar fazer o robô "olhar" para o site visualmente não ia funcionar a longo prazo (já que o site muda sempre). Eu instruí a IA a mudar a estratégia: em vez de ler a tela, o robô passou a "ouvir" a conexão de internet do site, capturando os dados das vagas diretamente da fonte (no formato de texto), antes mesmo de eles aparecerem na tela. Isso deixou a busca incrivelmente rápida e imune a travamentos visuais.

### 2. O Problema da IA "Faladeira" e Fora do Padrão

- **O que eu percebi testando:** Ao mandar as vagas para o Google Gemini avaliar, a IA devolvia respostas cheias de firulas, textos amigáveis ("Olá, Thiago, aqui está a sua vaga!") e formatações que quebravam a minha planilha do Google Sheets. Além disso, ela se confundia e dava notas altas para vagas que eu não queria.
- **A minha solução:** Eu atuei criando regras de negócio super rígidas. Escrevi um comando direto para a IA dizendo: *"Aja como um recrutador técnico e corte a conversa"*. Eu mesmo defini uma escala de 0 a 100 e criei exemplos práticos para ela: mostrei que vagas de Análise de Dados e Automação deveriam ter nota máxima, mas vagas de Suporte ou Atendimento (mesmo que eu tenha experiência) deveriam ser penalizadas com nota baixa, pois quero fazer transição de carreira. Por fim, travei a criatividade da IA (usando um recurso chamado Temperatura Zero), obrigando-a a devolver apenas o número da nota e 3 linhas de justificativa.

### 3. O Problema das Falhas em Cascata e o Limite da API

- **O que eu percebi testando:** O meu Google Sheets tem dezenas de vagas. Se o robô estivesse na vaga número 15 e a internet oscilasse, ou a IA demorasse para responder, o programa inteiro dava erro e parava. Além disso, ao mandar muitas vagas de uma vez, eu atingia o limite gratuito de perguntas por minuto do Google.
- **A minha solução:** Eu instruí a criação de um "isolamento de risco" e uma fila de espera. Primeiro, pedi para colocar um intervalo de 20 segundos de pausa entre cada avaliação de vaga para respeitar os limites do Google. Segundo, criei uma regra de segurança: *"Se a inteligência artificial falhar em uma vaga específica, não pare o sistema. Apenas anote 'Erro' nessa vaga, dê nota 0 por segurança e continue a avaliação da próxima vaga"*. Com isso, o sistema passou a rodar do início ao fim sem precisar da minha supervisão.

#### 4. O Problema da Segurança das Senhas

- **O que eu percebi testando:** Para conectar o sistema com o meu banco de dados (Google Sheets) e com o meu cérebro de inteligência artificial, eu precisei gerar chaves de acesso. Inicialmente, essas senhas estavam soltas no meio do código do robô.
- **A minha solução:** Pensando em transformar esse projeto em um portfólio público, percebi que não poderia colocar meu código na internet com essas chaves expostas. Instruí a IA a separar toda a parte de senhas e credenciais em arquivos trancados (isolados), criando um escudo (gitignore) para garantir que, quando qualquer pessoa baixe o meu projeto para ver como eu fiz, ela veja apenas a lógica do robô, mantendo meus dados e minha nuvem 100% protegidos.